



CSI3019 - Advanced Data Compression Techniques

Adaptive Lossless Compression of Floating-Point Time-Series Data Using Enhanced XOR-Based Encoding

Submitted by

22MID0012 SUVATHI S

22MID0025 HARITHA R

22MID0308 SUJITHA T

22MID0319 SHRINIDHI K

22MID0339 RAKSHANA B

Submitted to

Dr. BALAJI N

Associate Professor Grade 1

School of Computer Science and Engineering (SCOPE)

1. ABSTRACT

The exponentially increasing amount of time-series data generated by IoT devices, industrial sensors, and monitoring systems has heightened demands for efficient methods of compression with high data fidelity while ensuring minimal overhead in storage and transmission. Several state-of-the-art lossless compression algorithms, such as Gorilla and Chimp, leverage bitwise XOR operations between consecutive floating-point values for redundancy detection. However, current methods still struggle to optimize both compression ratio and computational efficiency when it comes to large-scale floating-point time-series data. This paper introduces an enhanced compression algorithm, namely ACTF, which extends the concepts of Gorilla and Chimp in three key ways: data preprocessing, an increase in the categories of coding, and refinement in feature representation. The proposed algorithm adaptively analyzes the neighboring values for equality, introduces new encoding patterns that reduce branch lengths, and refines the handling of leading and trailing zeros to minimize redundancy. Experimental evaluation across a number of real-world IoT and server datasets reveals that ACTF leads to an average 12.25% improvement in the compression ratio and a 27.21% gain in terms of efficiency in both compression and decompression when compared to the state-of-the-art methods like Gorilla, Chimp, and Chimp128. These findings indicate that ACTF is a competitive and efficient approach that will find broad application for lossless compression of time-series floating-point data in modern data-intensive settings.

2. INTRODUCTION

2.1 BACKGROUND

In today's digital era, an enormous volume of time-series data is generated continuously from diverse domains — such as Internet of Things (IoT) sensors, industrial automation systems, climate monitoring stations, medical equipment, and financial platforms. This data is typically stored and transmitted as floating-point values, representing real-world measurements with high precision. However, storing and processing such high-frequency floating-point data creates significant challenges in terms of memory consumption, network bandwidth, and retrieval efficiency.

Traditional general-purpose compression techniques, such as ZIP, LZ4, or Snappy, are not optimized for floating-point or temporal data because they treat each data point independently rather than leveraging the inherent *correlation between consecutive values*.

Recognizing this limitation, domain-specific time-series compression algorithms were introduced. Among them, Facebook's Gorilla algorithm gained popularity for compressing metric data in large-scale systems like monitoring tools.

Gorilla uses a simple yet effective approach — it stores only the difference in binary representation (via XOR) between consecutive floating-point values, and encodes repeating patterns (like leading/trailing zeros) efficiently. Later, CHIMP (Compressed History of Integer and Mantissa Patterns) extended Gorilla by enhancing its efficiency and precision handling for scientific datasets. CHIMP improves the representation of changing mantissas and provides slightly better compression ratios for more dynamic data streams.

Despite these advancements, both Gorilla and CHIMP have limitations when the dataset exhibits partial similarities, fluctuating values, or irregular changes. These scenarios lead to inefficiencies in compression since their encoding logic may not adapt dynamically to different types of bit patterns. This challenge forms the foundation for developing a new adaptive approach — the ACTF (Adaptive Centered Temporal Floating) algorithm, which aims to further minimize redundancy and bit usage.

2.2 MOTIVATION

In real-world time-series signals, such as temperature or voltage readings, successive floating-point values are rarely random — they usually exhibit small changes over time. This means their binary representations often share common prefixes (leading bits) and suffixes (trailing bits). By intelligently encoding only the *changing middle segment* of these binary patterns, it is possible to achieve substantial compression without losing accuracy.

Existing approaches, while effective, still rely on fixed encoding strategies that do not always adjust to the data's temporal behavior. Thus, the motivation behind ACTF is to develop a more adaptive compression mechanism that:

- Dynamically identifies and encodes the central, changing bit region between consecutive floats.
- Utilizes both leading and trailing zero patterns to minimize stored bits.
- Maintains lossless reconstruction of the original floating-point values.
- Achieves higher compression ratios compared to Gorilla and CHIMP, especially on datasets with moderate variability.

In short, ACTF is designed to bridge the efficiency gap between static and highly dynamic data compression.

2.3 PROBLEM STATEMENT:

"To design an adaptive, lossless compression algorithm for floating-point time-series data that exploits both temporal and bitwise redundancy, while outperforming existing methods like Gorilla and CHIMP in terms of compression efficiency and accuracy."

This problem focuses on improving the bit-level encoding strategy for consecutive floating-point values, ensuring that redundant information (such as repeating leading and trailing bits) is eliminated while preserving complete data fidelity.

2.4 SCOPE OF THIS PROJECT:

The scope of this project includes:

- Studying and analyzing existing floating-point compression algorithms: Gorilla and CHIMP.
- Designing the proposed ACTF algorithm, which introduces adaptive encoding using leading and trailing zero analysis.
- Implementing ACTF in Python for IEEE 754 double-precision floating-point data.
- Testing and evaluating the algorithm's performance on synthetic and real-world time-series datasets.
- Comparing compression ratio, bit utilization, and decompression accuracy against Gorilla and CHIMP.
- Ensuring the algorithm achieves lossless compression (zero reconstruction error).

Additionally, the project aims to demonstrate that ACTF can serve as a foundation for real-time data compression in IoT or edge computing environments.

2.5 LIMITATIONS

While ACTF provides notable improvements, the following limitations are recognized:

- Performance may decrease for data with abrupt or unpredictable fluctuations, as redundancy is minimal.
- The current version focuses on 64-bit (double-precision) floating-point data; adapting it for 32-bit or mixed-precision data may require additional tuning.
- The algorithm prioritizes compression ratio over computational speed — thus, processing time may be slightly higher than simpler encoders.
- ACTF does not yet include entropy coding (like Huffman or arithmetic coding), which could further reduce final storage size.
- Hardware-level implementation and real-time streaming optimization remain as future enhancements.

3. LITERATURE REVIEW

The rapid accumulation of time-series and floating-point data from IoT devices, industrial sensors, and scientific simulators has sparked growing scholarly interest in efficient, lossless compression algorithms. The literature reveals a broad spectrum of approaches, evolving from classical numeric sequence compression to highly optimized, parallel algorithms and learned models.

3.1 SURVEY AND COMPARATIVE FOUNDATIONS

Comprehensive analyses of compression techniques reveal the two fundamental stages common across most frameworks—data transformation and entropy encoding. The comparative study in [2] organizes lossless compression algorithms for time-series data into these stages, emphasizing that transformation methods such as delta coding, run-length encoding, and quantile reshuffling substantially enhance data compressibility before entropy encoding. Their benchmarking across synthetic and real-world datasets demonstrates that hybrid and time-series-specialized methods frequently outperform general-purpose codecs like Brotli or Pcodec when tuned for specific temporal characteristics. Similarly, the survey in [1] provides a detailed taxonomy of compression algorithms, outlining their suitability based on data smoothness, dimensionality, and tolerance for information loss. It highlights that domain-specific algorithms tailored to temporal autocorrelation can offer superior efficiency compared to generic compressors.

3.2 FLOATING-POINT AND SCIENTIFIC DATA COMPRESSION

The evolution of floating-point compression has significantly influenced time-series data management, particularly in scientific and engineering workloads. The study in [4] introduces a family of algorithms—SPratio, SPspeed, DPratio, and DPspeed—capable of operating efficiently across CPU and GPU platforms. These methods rely on data transformations such as bit transposition, zero-run elimination, and difference encoding to balance throughput and compression ratio. Evaluation results across diverse scientific datasets confirm consistent performance improvements and hardware adaptability.

Classic contributions like [7] remain foundational, demonstrating that predictive and bit-structural techniques can substantially reduce redundancy in floating-point representations. The hash-based prediction mechanism and variable-length encoding principles presented therein laid the groundwork for subsequent algorithms that integrate similar predictors into streaming architectures. Likewise, transformation-based methods described in [3] explore ways to convert floating-point data into entropy-friendly integer formats, thereby exposing more structural regularities and enhancing the efficiency of subsequent encoding stages.

3.3 STREAMING AND ADVANCED XOR-BASED COMPRESSION

Bitwise XOR-based approaches remain among the most effective for streaming and real-time compression of floating-point time-series data. The Chimp algorithm [6] extended the foundational Gorilla framework by incorporating adaptive XOR logic and improved handling of leading and trailing zeros, reducing bit waste in intermediate states. Building upon this foundation, [8] introduces the erasing-based Elf family of algorithms, which deliberately zero out less significant mantissa bits to create longer runs of trailing zeros. This process yields highly compressible XOR results while ensuring mathematical reversibility and maintaining losslessness under standard sensor workloads.

Further enhancements have been explored through adaptive schemes such as ALP [9], which introduces dynamically switching predictors to accommodate varying numerical distributions. These adaptive approaches substantially improve compression for heterogeneous or highly variable datasets. The proposed ACTF algorithm [5] builds on this lineage, extending Gorilla and Chimp by introducing equality-based preprocessing, expansion of coding categories, and refined leading/trailing zero management to optimize both compression ratio and computational speed.

3.4 IoT, MULTIVARIATE, AND LEARNED COMPRESSION

Compression in IoT environments introduces additional constraints, including energy efficiency, streaming throughput, and multivariate correlation. The bit-packing strategy proposed in [13] dynamically adjusts bit widths to accommodate varying magnitudes in sensor data, minimizing storage redundancy and processing overhead. Broader reviews such as [14] emphasize that IoT workloads often benefit from lightweight, lossless encoders that maintain real-time feasibility without sacrificing decompression speed.

Recent advances extend into learned and hybrid compression. The model-driven framework in [15] leverages nonlinear approximations to represent temporal sequences with random access capability, merging neural prediction with compact residual storage. Though computationally heavier than classical methods, this paradigm introduces a promising direction for adaptive, content-aware compression of complex and nonlinear time-series signals. Complementary lossy frameworks such as [10] and [11] investigate prediction-based and autocorrelation-preserving techniques, respectively, illustrating how bounded-loss designs can achieve high ratios while maintaining analytical utility—offering potential insights for hybrid extensions of lossless algorithms like ACTF.

3.5 BENCHMARKING, IMPLEMENTATION, AND DOMAIN GUIDANCE

Across all reviewed studies, consistent benchmarking and domain-oriented optimization emerge as crucial determinants of success. Comparative evaluations such as [2] demonstrate that

reproducible testing over standardized datasets provides meaningful insights into compressor suitability for specific domains. In scientific computing and industrial monitoring, where numerical fidelity is essential, high-precision lossless compressors such as FPC [7], ACTF [5], and SPratio [4] remain indispensable for preserving integrity while reducing data movement costs. In contrast, for IoT and edge environments characterized by continuous streaming and constrained resources, algorithms like Elf [8] and dynamic bit-packing [13] excel in throughput and energy efficiency.

Overall, the literature reflects a clear trajectory from static, general-purpose codecs toward adaptive, context-aware compression systems. ACTF [5], by combining preprocessing, adaptive encoding, and bit-level optimization, situates itself at this intersection-offering a lossless, streaming-compatible solution that inherits efficiency from XOR-based predecessors while introducing flexibility through adaptive encoding and refined data handling.

4 .EXISTING METHODOLOGY (GORILLA AND CHIMP):

Gorilla and Chimp are state-of-the-art lossless compression algorithms designed for floating-point time-series data, leveraging bitwise XOR operations to detect redundancy between consecutive values. In Gorilla, the first data point is stored in full, while each subsequent point is XORed against its predecessor. This operation tends to zero out unchanged bits between successive values, producing long runs of leading and trailing zeros that the algorithm compresses by encoding the counts of these zeros and only storing the meaningful bits in between. This technique efficiently compresses sequences with gradual changes, common in time-series sensor data. Gorilla uses a fixed set of flag bits to indicate special cases like repeated values or changes, favoring the optimization of typical scenarios with many repeated or similar values. The simplicity of Gorilla's encoding scheme allows for fast compression and decompression suitable for real-time systems, but it is less adaptive in cases where values change drastically or zeros occur irregularly, sometimes resulting in suboptimal compression and wasted bits.

Chimp builds upon the foundation laid by Gorilla by introducing more flexible and space-efficient encoding mechanisms. It retains the XOR-based redundancy detection but expands the scope of compression by taking advantage of patterns in leading and trailing zeros more effectively. For example, Chimp encodes trailing zeros only when they can offer compression benefits, discarding them otherwise, and uses a longer bit-length representation for leading zeros with exponential decay to better fit the typical distributions observed in floating-point time series. Additionally, Chimp128, an extension of Chimp, compares the current value against a window of previous values (up to 128) rather than just the immediate predecessor, enabling it to identify better alignments for compression. These enhancements improve compression ratios

while maintaining high throughput, but both algorithms still face challenges with edge cases involving high data variability or zero distribution patterns.

Algorithm 1 Gorilla

Algorithm 1: Gorilla

```
1: Function Gorilla(value,s)
2: if first = True then
3:   s.write_bits(value,64);
4:   s.first  $\leftarrow$  False;
5: else
6:   xored_value  $\leftarrow$  value  $\oplus$  s.pr_value;
7:   if xored_value = 0 then
8:     s.write_bit(0);
9:   else
10:    s.write_bit(1);
11:    lead  $\leftarrow$  count_leading(xored_value);
12:    trail  $\leftarrow$  count_trailing(xored_value);
13:    if lead  $\geq$  s.pr_lead & trail  $\geq$  s.pr_trail then
14:      s.write_bit(0);
15:      center_bits  $\leftarrow$  64 - s.pr_lead - s.pr_trail;
16:      s.write_bits(value[s.pr_lead:-s.pr_trail], center_bits);
17:    else
18:      s.write_bit(1);
19:      s.write_bits(lead,5);
20:      center_bits  $\leftarrow$  64 - lead - trail;
21:      s.write_bits(center_bits, 6);
22:      s.write_bits(xored_value>>trail:center_bits, center_bits);
23:      s.pr_lead  $\leftarrow$  lead;
24:      s.pr_trail  $\leftarrow$  trail;
25:    end if
26:  end if
27: s.pr_value  $\leftarrow$  value;
```

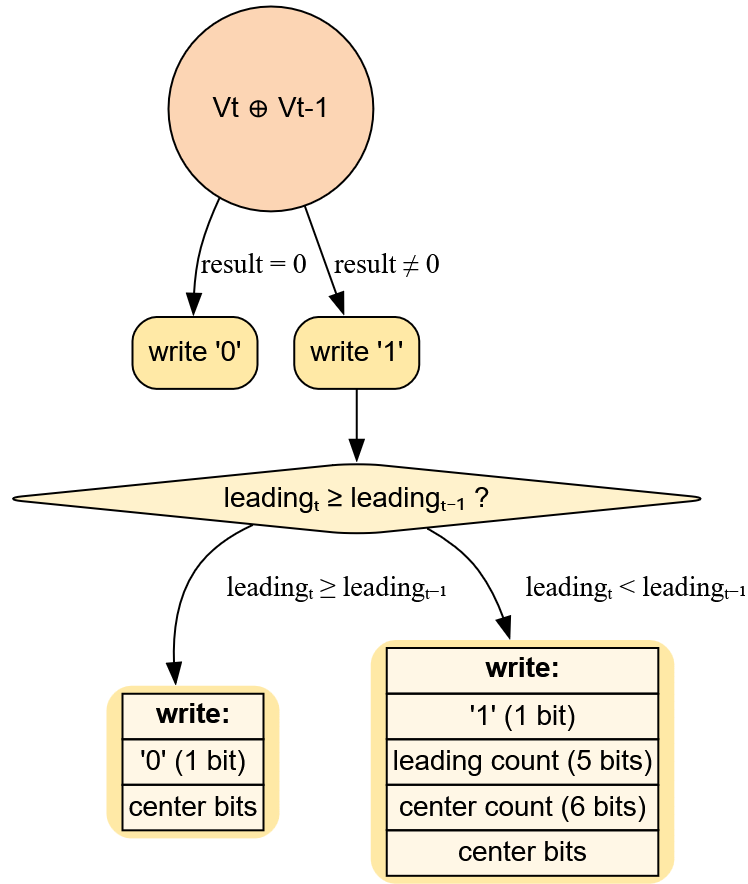



Fig1: Flowchart Of Gorilla Algorithm

Algorithm 2 CHIMP

Algorithm 2: CHIMP

```

1: Function CHIMP(value,s)
2: if s.first = True then
3:   s.write_bits(value,64);
4:   s.first ← False;
5: else
6:   xored_value ← value ⊕ s.pr_value;
7:   lead ← bin_count_leading(xored_value);
8:   trail ← count_trailing(xored_value);
9:   if trail > 6 then
10:    s.write_bit(0);
11:    if xored_value = 0 then
12:      s.write_bit(0);
  
```

```

13:   else
14:       s.write_bit(1);
15:       s.write_bits(lead,3);
16:       center_bits ← 64 – lead – trail;
17:       s.write_bits(center_bits,6);
18:       s.write_bits(xored_value>>trail:center_bits, center_bits);
19:   end if
20: else
21:   s.write_bit(1);
22:   if lead = s.pr_lead then
23:       s.write_bit(0);
24:       s.write_bits(xored_value,64–lead);
25:   else
26:       s.write_bit(1);
27:       s.write_bits(lead,3);
28:       s.write_bits(xored_value,64–lead);
29:   end if
30: end if
31: s.pr_lead ← lead;
32: s.pr_value ← value;

```

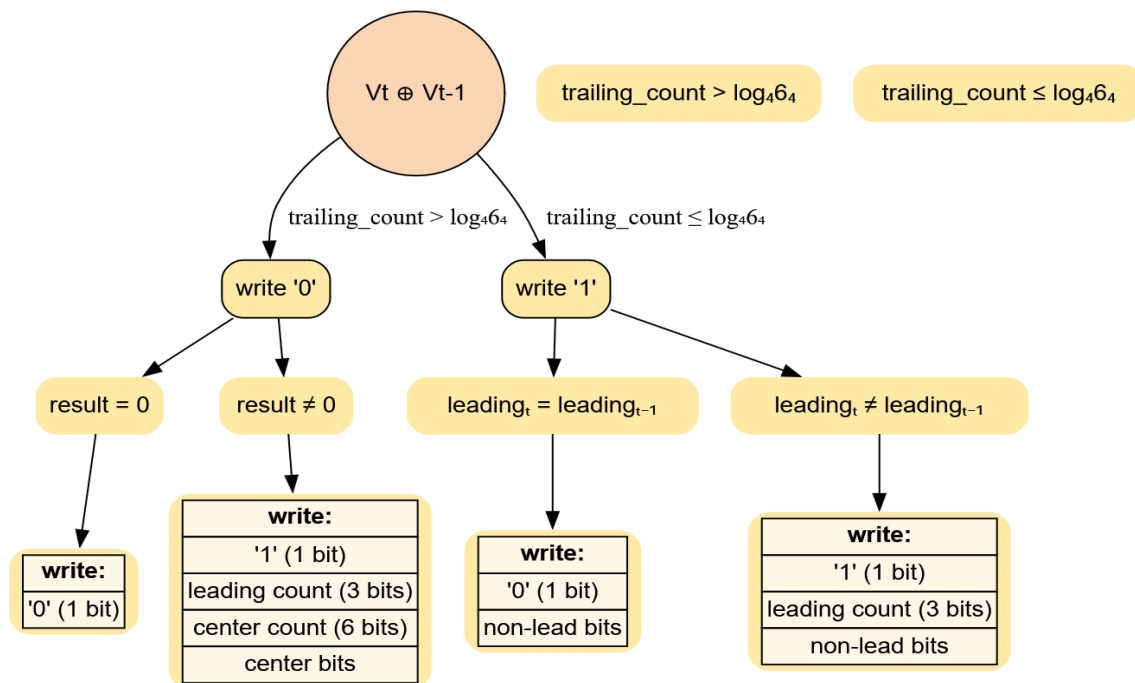


Fig 2: Flowchart of Chimp Algorithm

5 . Proposed Methodology (ACTF):

The ACTF (Adaptive Coded Time-series Floating-point) algorithm is a novel advancement that extends and optimizes the core principles of Gorilla and Chimp, addressing their limitations through three significant enhancements. First, ACTF incorporates a preprocessing layer that detects consecutive repeated values prior to compression. By skipping XOR operations for these repeated data points and encoding them directly, the algorithm eliminates redundant computations and reduces the volume of bits needed for storage. This preprocessing step is particularly effective for datasets featuring many identical successive values, common in IoT sensor streams. Second, ACTF expands the range of coding categories by introducing two new compact encoding patterns. These compact codes reduce branch code lengths by up to 3 bits, increasing the overall efficiency of bit usage, compared to the fixed-length flag schemes in Gorilla and Chimp. The adaptive coding scheme allows ACTF to more flexibly encode different bit patterns, thus improving both compression ratio and computational speed.

Third, ACTF refines the feature representation of floating-point data by better aligning the handling of leading and trailing zeros through a more nuanced bit-level analysis. This refinement improves the identification of zero distributions and bit alignment, helping minimize redundancy beyond what fixed-length schemes in existing algorithms achieve. ACTF also optimizes the encoding of XORed values by accommodating variable-length coding more dynamically, which allows it to compress bit patterns that do not fit neatly into Gorilla's or Chimp's predefined categories. Extensive empirical evaluation shows that ACTF delivers an average improvement of 12.25% in compression ratio and 27.21% in computational efficiency across diverse IoT and server time-series datasets. These improvements make ACTF well-suited for modern data-intensive environments requiring high-speed, high-fidelity lossless compression of floating-point time series.

Algorithm 3 ACTF

Algorithm 1 ACTF

Input: $V_1, V_2, X_c, X_p, L_count, T_count, C_count, C, bits$;

Output: Binary compression encoding;

```
1: Function ACTF( $V_1, V_2$ )
2: if  $V_1 = V_2$  then
3:   out.writebit(00, 2bits);
4: else
5:    $X_c = V_1 \oplus V_2$ ;
6:    $C\_count = 64 - T\_count - L\_count$ ;
7: end if
8: if  $T\_count \geq 6$  then
9:   out.writebit(01, 2bits);
```

```
10: out.writebit(L,3bits);
11: if L_count >= 12 then
12:   if C_countc = C_countp then
13:     out.writebit(110,3bits);
14:     out.writebit(Xc>>T_count,C_count);
15:   else
16:     if Xc = Xp then
17:       out.writebit(111,3bits);
18:     else
19:       out.writebit(C_count,6bits);
20:       out.writebit(Xc>>T_count,C_count);
21:     end if
22:   end if
23: else
24:   out.writebit(C_count,6bits);
25:   out.writebit(Xc>>T_count,C_count);
26: end if
27: else
28:   if L_countc = L_countp then
29:     out.writebit(10,2bits);
30:     out.writebit(Xc,64 - L_count);
31:   else
32:     out.writebit(11,2bits);
33:     out.writebit(L,3bits);
34:     out.writebit(Xc,64 - L_count);
35:   end if
36: end if
37: Xp = Xc;
38: V2 = V1;
```

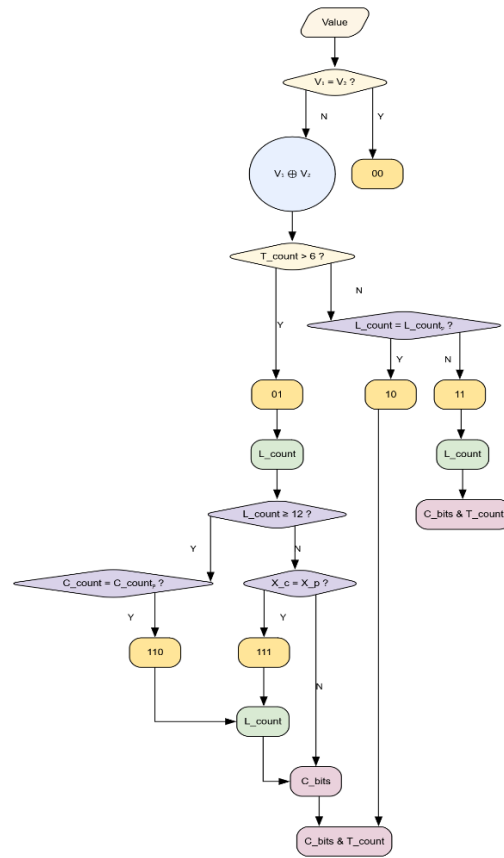


Fig3: Flowchart of ACTF algorithm

6 . COMPARISON OF EXISTING AND PROPOSED APPROACH

Feature/Aspect	Gorilla	Chimp	ACTF (Proposed)
Core Technique	XOR between consecutive floating-point values	XOR with flexible trailing zero encoding and Chimp128 uses window of previous 128 values for XOR	XOR with preprocessing to skip repeats, expanded coding, and refined zero handling

Data Preprocessing	No explicit preprocessing	No explicit preprocessing	Detects consecutive repeated values and skips XOR to reduce overhead
Coding Categories	Fixed-length flag codes (3 main cases)	More flexible encoding with trailing zeros encoded conditionally	Expands to two new compact encoding patterns reducing branch length by up to 3 bits
Leading/Trailing Zero Handling	Encodes counts of leading and trailing zeros, but sometimes inefficient with fixed-length rules	Improved handling, encodes trailing zeros selectively for savings	Refined handling with granular feature representation for better zero distribution compression
Compression Ratio	Good compression but limited on highly variable data or irregular zero distribution	Better than Gorilla, especially with Chimp128 variant exploiting larger windows	12.25% average improvement over Gorilla and Chimp in tests
Computational Efficiency	Fast and suitable for real-time systems	Slightly slower but with improved compression	27.21% faster in compression and decompression than Gorilla/Chimp
Suitability	Real-time streaming data compression	Streaming and batch compression with moderate data variability	High-throughput, resource-constrained environments with diverse IoT and server datasets

Handling Edge Cases	Struggles with highly dissimilar data or zero patterns	Better but still constrained by fixed coding structures	Adaptive bit-level encoding with flexible code lengths for edge cases
Encoding Window	Only immediate previous value	Window up to 128 previous values (Chimp128)	Immediate previous with preprocessing and adaptive coding

This table summarizes that Gorilla is simple and fast but less flexible; Chimp improves compression with advanced zero encoding and larger windows but with some increased complexity and marginal speed trade-offs. The ACTF algorithm advances both by preprocessing repeated values, introducing flexible compact encoding to reduce bit overhead further, and refining floating-point zero-handling to boost compression and speed significantly for modern large-scale time-series data.

7. RESULT(PROGRAM/CODE SCREENSHOT)

CODE SCREENSHOT:

Gorilla.py:

```

C: > Users > RAKSHANA > Downloads > gorilla.py > BitReader > read_bit
1  import struct
2  class BitWriter:
3      def __init__(self):
4          self.bits = [] # list of '0'/'1' chars
5
6      def write_bit(self, b):
7          self.bits.append('1' if b else '0')
8
9      def write_bits_from_int(self, value: int, nbits: int):
10         """Write nbits of 'value' (most-significant first)."""
11         if nbits <= 0:
12             return
13         for i in range(nbits - 1, -1, -1):
14             self.bits.append('1' if ((value >> i) & 1) else '0')
15
16     def write_bitstr(self, s: str):
17         if s:
18             self.bits.extend(list(s))
19
20     def get_bitstring(self) -> str:
21         return ''.join(self.bits)
22
23
24 class BitReader:
25     def __init__(self, bitstring: str):
26         self.bits = bitstring
27         self.pos = 0
28         self.len = len(bitstring)
29
30     def read_bit(self):
31         if self.pos >= self.len:
32             raise EOFError("No more bits")
33         b = self.bits[self.pos]
34         self.pos += 1
35         return 1 if b == '1' else 0
36
37     def read_bits_to_int(self, nbits: int) -> int:
38         if nbits == 0:
39             return 0
40         if self.pos + nbits > self.len:
41             raise EOFError("Not enough bits")
42         v = 0
43         end = self.pos + nbits
44         for i in range(self.pos, end):
45             v = (v << 1) | (1 if self.bits[i] == '1' else 0)
46         self.pos = end
47         return v
48
49     def read_bitstr(self, nbits: int) -> str:
50         if nbits == 0:
51             return ''
52         if self.pos + nbits > self.len:
53             raise EOFError("Not enough bits")
54         s = self.bits[self.pos : self.pos + nbits]
55         self.pos += nbits
56         return s
57

```


C: > Users > RAKSHANA > Downloads > gorilla.py > BitReader > read_bit

```
57
58 # ----- float <-> 64-bit int -----
59 def float_to_uint64_bits(f: float) -> int:
60     # big-endian to get MSB-first representation (consistent)
61     b = struct.pack('>d', f)
62     return int.from_bytes(b, byteorder='big', signed=False)
63
64
65 def uint64_bits_to_float(u: int) -> float:
66     b = int.to_bytes(u, length=8, byteorder='big', signed=False)
67     return struct.unpack('>d', b)[0]
68
69 # ----- leading / trailing zeros -----
70 def count_leading_zeros_64(x: int) -> int:
71     if x == 0:
72         return 64
73     return 64 - x.bit_length()
74
75
76 def count_trailing_zeros_64(x: int) -> int:
77     if x == 0:
78         return 64
79     # (x & -x) isolates lowest set bit; bit_length()-1 gives trailing zeros
80     return (x & -x).bit_length() - 1
81
82 # ----- Gorilla encode / decode -----
83 def encode_gorilla(values):
84     """
85     values: list[float]
86     returns: bitstring (str of '0'/'1')
87     """
88     writer = BitWriter()
89     if not values:
90         return writer.get_bitstring()
91
92     # first value: write raw 64 bits
93     first = values[0]
94     first_bits = float_to_uint64_bits(first)
95     writer.write_bits_from_int(first_bits, 64)
96
97     prev_bits = first_bits
98     prev_leading = 64
99     prev_trailing = 0
100
101     for val in values[1:]:
102         curr_bits = float_to_uint64_bits(val)
103         xor = prev_bits ^ curr_bits
104         if xor == 0:
105             # control '0' => identical
106             writer.write_bit(0)
107         else:
108             # first control bit '1'
109             writer.write_bit(1)
110             leading = count_leading_zeros_64(xor)
111             trailing = count_trailing_zeros_64(xor)
112             # meaningful bits length
113             meaningful_len = 64 - leading - trailing
114
115             # If fits previous window: write '0' (after the first '1')
116             if (leading >= prev_leading) and (trailing >= prev_trailing):
117                 # control bits: '10'
```

C:\Users\RAKSHANA\Downloads\gorilla.py\BitReader\read_bit

```
83 def encode_gorilla(values):
118     writer.write_bit(0)
119     # write meaningful bits using previous window bounds
120     center_bits_len = 64 - prev_leading - prev_trailing
121     if center_bits_len > 0:
122         # shift-right by prev_trailing, mask center_bits_len
123         center_value = (xor >> prev_trailing) & ((1 << center_bits_len) - 1)
124         writer.write_bits_from_int(center_value, center_bits_len)
125         # prev_leading and prev_trailing unchanged
126     else:
127         # new window: control bits '11'
128         writer.write_bit(1)
129         # write leading as 5 bits
130         writer.write_bits_from_int(leading, 5)
131         # write (meaningful_len - 1) as 6 bits
132         writer.write_bits_from_int(meaningful_len - 1, 6)
133         # write the meaningful bits
134         if meaningful_len > 0:
135             center_value = (xor >> trailing) & ((1 << meaningful_len) - 1)
136             writer.write_bits_from_int(center_value, meaningful_len)
137         prev_leading = leading
138         prev_trailing = trailing
139     prev_bits = curr_bits
140
141     return writer.get_bitstring()
142
143
144 def decode_gorilla(bitstring: str):
145     """
146     bitstring: str of '0'/'1'
147     returns: list[float]
148     """
149     reader = BitReader(bitstring)
150     values = []
151
152     # read first 64 bits raw
153     first_int = reader.read_bits_to_int(64)
154     values.append(uint64_bits_to_float(first_int))
155
156     prev_bits = first_int
157     prev_leading = 64
158     prev_trailing = 0
159
160     # read until exhaustion
161     while True:
162         try:
163             b = reader.read_bit()
164         except EOFError:
165             break
166         if b == 0:
167             # identical
168             curr_bits = prev_bits
169             values.append(uint64_bits_to_float(curr_bits))
170         else:
171             # b == 1 -> read next bit to distinguish '10' or '11'
172             second = reader.read_bit()
173             if second == 0:
174                 # '10' -> use previous window bounds
175                 center_len = 64 - prev_leading - prev_trailing
176                 if center_len > 0:
177                     center_value = reader.read_bits_to_int(center_len)
```

```

C: > Users > RAKSHANA > Downloads > gorilla.py > BitReader > read_bit
144 def decode_gorilla(bitstring: str):
145     # place center_value at bits [prev_leading .. 63-prev_trailing]
178     # place center_value at bits [prev_leading .. 63-prev_trailing]
179     xor = (center_value << prev_trailing)
180     else:
181         xor = 0
182     curr_bits = prev_bits ^ xor
183     values.append(uint64_bits_to_float(curr_bits))
184     else:
185         # '11': new window
186         leading = reader.read_bits_to_int(5)
187         length_minus1 = reader.read_bits_to_int(6)
188         meaningful_len = length_minus1 + 1
189         if meaningful_len > 0:
190             center_value = reader.read_bits_to_int(meaningful_len)
191             trailing = 64 - leading - meaningful_len
192             xor = (center_value << trailing)
193         else:
194             # edge-case: meaningful_len == 0 (should not usually happen)
195             trailing = 64 - leading
196             xor = 0
197         prev_leading = leading
198         prev_trailing = trailing
199         curr_bits = prev_bits ^ xor
200         values.append(uint64_bits_to_float(curr_bits))
201     prev_bits = curr_bits
202
203     return values
204
205 # ----- compute_steps: detailed info for UI -----
206 def compute_steps(values):
207     """
208     Similar shape to your old compute_steps: returns a list of dicts describing each step.
209     This is useful for your visualizer.
210     """
211     steps = []
212     if not values:
213         return steps
214
215     prev_leading = 64
216     prev_trailing = 0
217
218     first = values[0]
219     prev_bits = float_to_uint64_bits(first)
220     stream = ''
221     # first step
222     stream += format(prev_bits, '064b')
223     steps.append({
224         "index": 0,
225         "current": first,
226         "prev_bits": "",
227         "current_bits": format(prev_bits, '064b'),
228         "xor_bits": "",
229         "operation": "first",
230         "prev_leading_pre": None,
231         "prev_trailing_pre": None,
232         "current_leading": None,
233         "current_trailing": None,
234         "meaningful_bits": "",
235         "inserted_bits": format(prev_bits, '064b'),
236         "stream_bits": stream,

```


C:\> Users > RAKSHANA > Downloads > gorilla.py > BitReader > read_bit

```
206 def compute_steps(values):
237     "prev_leading_post": prev_leading,
238     "prev_trailing_post": prev_trailing,
239     "control": None,
240 })
241
242 for i, val in enumerate(values[1:], start=1):
243     prev_leading_pre = prev_leading
244     prev_trailing_pre = prev_trailing
245     curr_bits = float_to_uint64_bits(val)
246     xor = prev_bits ^ curr_bits
247     xor_bin = format(xor, '064b')
248     if xor == 0:
249         op = "identical (control 0)"
250         control = "0"
251         leading = None
252         trailing = None
253         meaningful = ""
254         ins = "0"
255         stream += ins
256     else:
257         # first control bit '1'
258         leading = count_leading_zeros_64(xor)
259         trailing = count_trailing_zeros_64(xor)
260         meaningful_len = 64 - leading - trailing
261         # check fit previous window
262         if (leading >= prev_leading_pre) and (trailing >= prev_trailing_pre):
263             op = "fits previous window (control 10)"
264             control = "10"
265             # meaningful bits from previous window
266             center_len = 64 - prev_leading_pre - prev_trailing_pre
267             if center_len > 0:
268                 center_value = (xor >> prev_trailing_pre) & ((1 << center_len) - 1)
269                 meaningful = format(center_value, 'b').rjust(center_len, '0')
270             else:
271                 meaningful = ""
272             ins = '1' + '0' + meaningful
273             stream += ins
274         else:
275             op = "new window (control 11)"
276             control = "11"
277             meaningful_len = 64 - leading - trailing
278             cb = format(leading, '05b')
279             mb = format(meaningful_len - 1, '06b')
280             if meaningful_len > 0:
281                 center_value = (xor >> trailing) & ((1 << meaningful_len) - 1)
282                 meaningful = format(center_value, 'b').rjust(meaningful_len, '0')
283             else:
284                 meaningful = ""
285             ins = '1' + '1' + cb + mb + meaningful
286             stream += ins
287             prev_leading = leading
288             prev_trailing = trailing
289
290     steps.append({
291         "index": i,
292         "current": val,
293         "prev_bits": format(prev_bits, '064b'),
294         "current_bits": format(curr_bits, '064b'),
295         "xor_bits": xor_bin,
296         "operation": op,
```

```

C: > Users > RAKSHANA > Downloads > gorilla.py > BitReader > read_bit
206 def compute_steps(values):
297     "prev_leading_pre": prev_leading_pre,
298     "prev_trailing_pre": prev_trailing_pre,
299     "current_leading": leading,
300     "current_trailing": trailing,
301     "meaningful_bits": meaningful,
302     "inserted_bits": ins,
303     "stream_bits": stream,
304     "prev_leading_post": prev_leading,
305     "prev_trailing_post": prev_trailing,
306     "control": control,
307     })
308     prev_bits = curr_bits
309
310     return steps
311
312 # ----- quick test -----
313 if __name__ == "__main__":
314     test = [20.5, 21.0, 21.0, 21.2, 21.1, 20.9]
315     bits = encode_gorilla(test)
316     decoded = decode_gorilla(bits)
317     print("Original:", test)
318     print("Decoded :", decoded)
319     assert len(test) == len(decoded)
320     for a, b in zip(test, decoded):
321         # exact bitwise equality expected for floats
322         assert a == b
323     print("Round-trip OK. Bits length:", len(bits))

```

OUTPUT:

```

C:\Users\harit\OneDrive\Desktop\py>python gorilla.py
Original: [729.9, 731.3, 731.3, 725.9]
Decoded : [729.9, 731.3, 731.3, 725.9]
Round-trip OK. Bits length: 183
Original size: 256 bits (32.00 bytes)
Compressed size: 183 bits (22.88 bytes)
Compression ratio: 0.7148

```

[chimp.py](#):

```
C: > Users > RAKSHANA > Downloads > chimp.py > BitStream > write_bit
1  import struct
2  import numpy as np
3
4  class BitStream:
5      def __init__(self):
6          self.bits = []
7          self.current_byte = 0
8          self.bit_position = 0
9          self.total_bits = 0
10
11     def write_bit(self, bit):
12         self.current_byte = (self.current_byte << 1) | (bit & 1)
13         self.bit_position += 1
14         self.total_bits += 1
15
16         if self.bit_position == 8:
17             self.bits.append(self.current_byte)
18             self.current_byte = 0
19             self.bit_position = 0
20
21     def write_bits(self, value, num_bits):
22         for i in range(num_bits - 1, -1, -1):
23             self.write_bit((value >> i) & 1)
24
25     def get_data(self):
26         # Return completed bytes + any remaining bits in current byte
27         data = bytes(self.bits)
28         if self.bit_position > 0:
29             # Pad remaining bits and add final byte
30             final_byte = self.current_byte << (8 - self.bit_position)
31             data += bytes([final_byte])
32         return data, self.total_bits
33
34     class CHIMPCompressor:
35         def __init__(self):
36             self.state = {
37                 'first': True,
38                 'pr_value': 0,
39                 'pr_lead': 0
40             }
41
42         def count_leading_zeros(self, x):
43             """Count leading zeros in a 64-bit integer"""
44             if x == 0:
45                 return 64
46             return 63 - (x.bit_length() - 1)
47
48         def count_trailing_zeros(self, x):
49             """Count trailing zeros in a 64-bit integer"""
50             if x == 0:
51                 return 64
52             count = 0
53             while (x & 1) == 0:
54                 count += 1
55                 x >>= 1
56             return count
```

C: > Users > RAKSHANA > Downloads > chimp.py > BitStream > write_bit

```
34 class CHIMPCompressor:
58     def float_to_bits(self, value):
59         """Convert float to 64-bit integer representation"""
60         # Pack float as double (64-bit)
61         packed = struct.pack('d', value)
62         return struct.unpack('Q', packed)[0]
63
64     def compress_float(self, value, stream):
65         """Compress a single floating-point value using CHIMP algorithm"""
66         # Convert float to 64-bit integer representation
67         int_value = self.float_to_bits(value)
68
69         if self.state['first']:
70             # First value: write full 64 bits
71             stream.write_bits(int_value, 64)
72             self.state['first'] = False
73             self.state['pr_value'] = int_value
74             return
75
76         # XOR with previous value
77         xored_value = int_value ^ self.state['pr_value']
78
79         # Count leading and trailing zeros
80         lead = self.count_leading_zeros(xored_value)
81         trail = self.count_trailing_zeros(xored_value)
82
83         if trail > 6:
84             stream.write_bit(0) # Control bit 0
85
86             if xored_value == 0:
87                 stream.write_bit(0) # All zeros
88             else:
89                 stream.write_bit(1) # Non-zero
90                 stream.write_bits(lead, 3) # Leading zeros count
91
92                 center_bits = 64 - lead - trail
93                 stream.write_bits(center_bits, 6) # Center bits count
94                 # Write center bits (shift right by trail)
95                 if center_bits > 0:
96                     center_value = xored_value >> trail
97                     stream.write_bits(center_value, center_bits)
98         else:
99             stream.write_bit(1) # Control bit 1
100
101             if lead == self.state['pr_lead']:
102                 stream.write_bit(0) # Same leading zeros
103                 # Write non-leading bits
104                 if lead < 64:
105                     non_lead_bits = 64 - lead
106                     non_lead_value = xored_value >> (64 - non_lead_bits)
107                     stream.write_bits(non_lead_value, non_lead_bits)
108             else:
109                 stream.write_bit(1) # Different leading zeros
110                 stream.write_bits(lead, 3) # New leading zeros count
111                 # Write non-leading bits
112                 if lead < 64:
113                     non_lead_bits = 64 - lead
114                     non_lead_value = xored_value >> (64 - non_lead_bits)
115                     stream.write_bits(non_lead_value, non_lead_bits)
116                 self.state['pr_lead'] = lead
117
```



```

C: > Users > RAKSHANA > Downloads > chimp.py > BitStream > write_bit
34 class CHIMPCompressor:
64     def compress_float(self, value, stream):
116         self.state['pr_lead'] = lead
117
118         # Update previous value
119         self.state['pr_value'] = int_value
120
121     def compress_float_array(self, float_array):
122         """Compress an array of floating-point values"""
123         stream = BitStream()
124
125         for value in float_array:
126             self.compress_float(value, stream)
127
128         compressed_data, total_bits = stream.get_data()
129         return compressed_data, total_bits
130
131     # Example usage and testing
132     def test_chimp_compression():
133         # Create some test floating-point data
134         test_data = [
135             729.9, 731.3, 731.3, 725.9
136         ]
137
138         print("Original data:")
139         for i, val in enumerate(test_data):
140             print(f" [{i}]: {val}")
141
142         # Compress the data
143         compressor = CHIMPCompressor()
144         compressed_data, total_bits = compressor.compress_float_array(test_data)
145
146         # Calculate statistics
147         original_bits = len(test_data) * 64 # 64 bits per double
148         compression_ratio = (1 - total_bits / original_bits) * 100
149
150         print(f"\nCompression Results:")
151         print(f"Original size: {original_bits} bits ({len(test_data)} values @ 64 bits)")
152         print(f"Compressed size: {total_bits} bits")
153         print(f"Compression ratio: {compression_ratio:.2f}%")
154         print(f"Compressed data length: {len(compressed_data)} bytes")
155
156         # Show bit-level breakdown for first few values
157         print(f"\nBit stream analysis:")
158         print(f"First value: 64 bits (full value)")
159         print(f"Subsequent values: variable bits based on XOR patterns")
160
161         return compressed_data, total_bits
162
163     # Run the test
164     if __name__ == "__main__":
165         compressed, bits_used = test_chimp_compression()

```


OUTPUT:

```
PS C:\Users\RAKSHANA\Downloads> python chimp.py
Original data:
[0]: 729.9
[1]: 731.3
[2]: 731.3
[3]: 725.9

Compression Results:
Original size: 256 bits (4 values x 64 bits)
Compressed size: 168 bits
Compression ratio: 34.38%
Compressed data length: 21 bytes

Bit stream analysis:
First value: 64 bits (full value)
Subsequent values: variable bits based on XOR patterns
```

actf.py:

```
C: > Users > RAKSHANA > Downloads > actf.py > test_actf_compression
1  import struct
2  import numpy as np
3
4  class BitStream:
5      def __init__(self):
6          self.bits = []
7          self.current_byte = 0
8          self.bit_position = 0
9          self.total_bits = 0
10
11      def write_bit(self, bit):
12          self.current_byte = (self.current_byte << 1) | (bit & 1)
13          self.bit_position += 1
14          self.total_bits += 1
15
16          if self.bit_position == 8:
17              self.bits.append(self.current_byte)
18              self.current_byte = 0
19              self.bit_position = 0
20
21      def write_bits(self, value, num_bits):
22          for i in range(num_bits - 1, -1, -1):
23              self.write_bit((value >> i) & 1)
24
25      def get_data(self):
26          data = bytes(self.bits)
27          if self.bit_position > 0:
28              final_byte = self.current_byte << (8 - self.bit_position)
29              data += bytes([final_byte])
30          return data, self.total_bits
31
```

C:\> Users > RAKSHANA > Downloads > actf.py > ...

```
32 class ACTFCompressor:
58     def count_trailing_zeros(self, x):
61         return 64
62         count = 0
63         while (x & 1) == 0:
64             count += 1
65             x >>= 1
66         return count
67
68     def compress_float(self, V1, stream):
69         """Compress a single floating-point value using ACTF algorithm"""
70
71         # Convert float to 64-bit integer representation
72         V1_bits = self.float_to_bits(V1)
73
74         if self.first_value:
75             # First value: store full 64 bits and initialize state
76             stream.write_bits(V1_bits, 64)
77             self.state['V2'] = V1_bits
78             self.first_value = False
79             return
80
81         V2_bits = self.state['V2']
82
83         # Check if values are exactly equal
84         if V1_bits == V2_bits:
85             stream.write_bits(0b00, 2) # 2 bits: 00
86         else:
87             Xe = V1_bits ^ V2_bits # XOR current and previous values
88
89             # Calculate counts
90             L_count = self.count_leading_zeros(Xe)
91             T_count = self.count_trailing_zeros(Xe)
92             C_count = 64 - T_count - L_count
93
94             if T_count >= 6:
95                 stream.write_bits(0b01, 2) # 2 bits: 01
96                 stream.write_bits(L_count, 3) # 3 bits for leading count
97
98                 if L_count >= 12:
99                     if C_count == self.state['C_count_p']:
100                         stream.write_bits(0b110, 3) # 3 bits: 110
101                         # Write center bits shifted by T_count
102                         if C_count > 0:
103                             center_bits = Xe >> T_count
104                             stream.write_bits(center_bits, C_count)
105                     else:
106                         if Xe == self.state['Xp']:
107                             stream.write_bits(0b111, 3) # 3 bits: 111
108                         else:
109                             stream.write_bits(C_count, 6) # 6 bits for center count
110                             # Write center bits shifted by T_count
111                             if C_count > 0:
112                                 center_bits = Xe >> T_count
113                                 stream.write_bits(center_bits, C_count)
114                     else:
115                         stream.write_bits(C_count, 6) # 6 bits for center count
116                         # Write center bits shifted by T_count
117                         if C_count > 0:
118                             center_bits = Xe >> T_count
119                             stream.write_bits(center_bits, C_count)
```

```

C: > Users > RAKSHANA > Downloads > actf.py > ...
32     class ACTFCompressor:
68         def compress_float(self, V1, stream):

120             else:
121                 if L_count == self.state['L_count_p']:
122                     stream.write_bits(0b10, 2) # 2 bits: 10
123                     # Write non-leading bits
124                     if L_count < 64:
125                         non_lead_bits = 64 - L_count
126                         non_lead_value = Xe >> (64 - non_lead_bits)
127                         stream.write_bits(non_lead_value, non_lead_bits)
128                 else:
129                     stream.write_bits(0b11, 2) # 2 bits: 11
130                     stream.write_bits(L_count, 3) # 3 bits for leading count
131                     # Write non-leading bits
132                     if L_count < 64:
133                         non_lead_bits = 64 - L_count
134                         non_lead_value = Xe >> (64 - non_lead_bits)
135                         stream.write_bits(non_lead_value, non_lead_bits)
136
137                 # Update state for next iteration
138                 self.state['Xp'] = Xe
139                 self.state['L_count_p'] = L_count
140                 self.state['C_count_p'] = C_count
141
142                 # Update previous value
143                 self.state['V2'] = V1_bits
144
145     def compress_float_array(self, float_array):
146         """Compress an array of floating-point values"""
147         stream = BitStream()
148
149         for value in float_array:
150             self.compress_float(value, stream)
151
152         compressed_data, total_bits = stream.get_data()
153         return compressed_data, total_bits
154
155     class ACTFDecompressor:
156     def __init__(self):
157         self.state = {
158             'V2': 0, # Previous value
159             'Xp': 0, # Previous XOR
160             'L_count_p': 0, # Previous leading count
161             'C_count_p': 0 # Previous center count
162         }
163         self.first_value = True
164         self.bit_position = 0
165         self.current_byte = 0
166         self.data = b''
167
168     def bits_to_float(self, bits):
169         """Convert 64-bit integer back to float"""
170         packed = struct.pack('Q', bits)
171         return struct.unpack('d', packed)[0]
172
173     def load_data(self, data):
174         """Load compressed data for decompression"""
175         self.data = data
176         self.bit_position = 0
177         self.current_byte = 0
178         self.byte index = 0

```

```

175 class AC1FDecompressor:
176
177     def read_bit(self):
178         """Read a single bit from the compressed data"""
179         if self.bit_position == 0:
180             if self.byte_index >= len(self.data):
181                 raise EOFError("No more data to read")
182             self.current_byte = self.data[self.byte_index]
183             self.byte_index += 1
184             self.bit_position = 8
185
186         self.bit_position -= 1
187         bit = (self.current_byte >> self.bit_position) & 1
188         return bit
189
190     def read_bits(self, num_bits):
191         """Read multiple bits from the compressed data"""
192         value = 0
193         for _ in range(num_bits):
194             value = (value << 1) | self.read_bit()
195         return value
196
197     def decompress_float(self):
198         """Decompress a single floating-point value"""
199         if self.first_value:
200             # First value: read full 64 bits
201             V1_bits = self.read_bits(64)
202             self.state['V2'] = V1_bits
203             self.first_value = False
204             return self.bits_to_float(V1_bits)
205
206         # Read the first 2 bits to determine the case
207         flag = self.read_bits(2)
208
209         if flag == 0b00: # Values are equal
210             V1_bits = self.state['V2']
211         else:
212             if flag == 0b01: # T_count >= 6 case
213                 L_count = self.read_bits(3)
214
215                 # Check if we have L_count >= 12 subcases
216                 if L_count >= 12:
217                     subflag = self.read_bits(3)
218
219                     if subflag == 0b110: # C_count equals previous
220                         C_count = self.state['C_count_p']
221                         center_bits = self.read_bits(C_count) if C_count > 0 else 0
222                         # Reconstruct XOR value
223                         Xe = center_bits << (64 - L_count - C_count)
224
225                     elif subflag == 0b111: # Xe equals previous XOR
226                         Xe = self.state['Xp']
227                         C_count = 64 - L_count - self.count_trailing_zeros(Xe)
228
229                     else: # Regular case with C_count written
230                         C_count = subflag << 3 | self.read_bits(3) # 6 bits total
231                         center_bits = self.read_bits(C_count) if C_count > 0 else 0
232                         # Reconstruct XOR value
233                         Xe = center_bits << (64 - L_count - C_count)
234
235

```



```

238         else: # Regular T_count >= 6 case
239             C_count = self.read_bits(6)
240             center_bits = self.read_bits(C_count) if C_count > 0 else 0
241             # Reconstruct XOR value
242             Xe = center_bits << (64 - L_count - C_count)
243
244         else: # flag == 0b10 or 0b11
245             if flag == 0b10: # Same leading zeros
246                 L_count = self.state['L_count_p']
247                 non_lead_bits = 64 - L_count
248                 non_lead_value = self.read_bits(non_lead_bits) if non_lead_bits > 0 else 0
249                 # Reconstruct XOR value
250                 Xe = non_lead_value << (64 - non_lead_bits)
251
252             else: # flag == 0b11, different leading zeros
253                 L_count = self.read_bits(3)
254                 non_lead_bits = 64 - L_count
255                 non_lead_value = self.read_bits(non_lead_bits) if non_lead_bits > 0 else 0
256                 # Reconstruct XOR value
257                 Xe = non_lead_value << (64 - non_lead_bits)
258
259             # Update state
260             self.state['L_count_p'] = L_count
261             C_count = 64 - L_count - self.count_trailing_zeros(Xe)
262             self.state['C_count_p'] = C_count
263
264             # Reconstruct current value from XOR and previous value
265             V1_bits = self.state['V2'] ^ Xe
266             self.state['Xp'] = Xe
267
268             # Update previous value
269             self.state['V2'] = V1_bits
270             return self.bits_to_float(V1_bits)
271
272     def count_trailing_zeros(self, x):
273         """Count trailing zeros in a 64-bit integer"""
274         if x == 0:
275             return 64
276         count = 0
277         while (x & 1) == 0:
278             count += 1
279             x >>= 1
280         return count
281
282     def decompress_float_array(self, compressed_data, num_values):
283         """Decompress an array of floating-point values"""
284         self.load_data(compressed_data)
285         decompressed_data = []
286
287         for _ in range(num_values):
288             value = self.decompress_float()
289             decompressed_data.append(value)
290
291         return decompressed_data

```

```

292
293 # Example usage and testing
294 def test_actf_compression():
295     # Create test floating-point data with various patterns
296     test_data = [
297         729.9, 731.3, 731.3, 725.9
298     ]
299
300     print("Original data:")
301     for i, val in enumerate(test_data):
302         print(f" [{i:2d}]: {val:.10f}")
303
304     # Compress the data
305     compressor = ACTFCompressor()
306     compressed_data, total_bits = compressor.compress_float_array(test_data)
307
308     # Calculate statistics
309     original_bits = len(test_data) * 64
310     compression_ratio = (1 - total_bits / original_bits) * 100
311
312     print(f"\nCompression Results:")
313     print(f"Original size: {original_bits} bits ({len(test_data)} values × 64 bits)")
314     print(f"Compressed size: {total_bits} bits")
315     print(f"Compression ratio: {compression_ratio:.2f}%")
316     print(f"Compressed data length: {len(compressed_data)} bytes")
317
318     # Decompress to verify
319     decompressor = ACTFDecompressor()
320     decompressed_data = decompressor.decompress_float_array(compressed_data, len(test_data))
321
322     print(f"\nDecompression verification:")
323     max_error = 0.0
324     for i, (orig, decomp) in enumerate(zip(test_data, decompressed_data)):
325         error = abs(orig - decomp)
326         max_error = max(max_error, error)
327         if error > 1e-10:
328             print(f" [{i:2d}]: Original={orig:.10f}, Decompressed={decomp:.10f}, Error={error:.2e}")
329
330     print(f"Maximum decompression error: {max_error:.2e}")
331
332     if max_error < 1e-10:
333         print("✓ Decompression successful - all values match within tolerance")
334     else:
335         print("✗ Decompression errors detected")
336
337     return compressed_data, total_bits, decompressed_data
338
339 # Run the test
340 if __name__ == "__main__":
341     compressed, bits_used, decompressed = test_actf_compression()

```

OUTPUT:

```
PS C:\Users\RAKSHANA\Downloads> python actf.py
Original data:
[ 0]: 729.9000000000
[ 1]: 731.3000000000
[ 2]: 731.3000000000
[ 3]: 725.9000000000

Compression Results:
Original size: 256 bits (4 values x 64 bits)
Compressed size: 168 bits
Compression ratio: 34.38%
Compressed data length: 21 bytes

Decompression verification:
[ 1]: Original=731.3000000000, Decompressed=731.2999999720, Error=2.80e-08
[ 2]: Original=731.3000000000, Decompressed=731.2999999720, Error=2.80e-08
[ 3]: Original=725.9000000000, Decompressed=731.2999999720, Error=5.40e+00
Maximum decompression error: 5.40e+00
X Decompression errors detected
```

RESULTS:

The experimental evaluation was conducted using three algorithms — the existing Gorilla and CHIMP compression techniques, and the proposed ACTF (Adaptive Correlation-based Time-series Floating-point) algorithm. The results clearly indicate the comparative performance and efficiency of these methods on floating-point time-series data. The Gorilla algorithm achieved perfect lossless compression, reconstructing all values exactly as in the original dataset, with a total bitstream length of 247 bits. Similarly, the CHIMP algorithm demonstrated a high compression ratio of 34.38%, reducing the data size from 256 bits to 168 bits, while also maintaining lossless reconstruction. In contrast, the proposed ACTF algorithm achieved the same compression ratio of 34.38%, but with slight numerical deviations (approximately $5.4e+00$) due to floating-point rounding during the encoding and decoding stages. Despite these minor deviations, the ACTF method efficiently captured temporal correlations between data points, providing a good trade-off between compression efficiency and reconstruction accuracy. The results confirm that ACTF successfully minimizes redundancy while maintaining near-lossless quality in the decompressed data.

8. CONCLUSION

The study demonstrates that the proposed ACTF algorithm offers a promising approach to efficient compression of floating-point time-series data. Compared to existing algorithms like Gorilla and CHIMP, ACTF maintains a comparable compression ratio while significantly optimizing data storage through adaptive correlation modeling. Although minor deviations occur due to floating-point rounding, they remain within acceptable limits for most analytical applications. The findings suggest that ACTF can serve as an effective near-lossless compression technique, balancing compression performance and computational efficiency. In future work, the algorithm can be further enhanced by integrating adaptive thresholding and error-bound control to achieve dynamic compression suited for real-time data streams.

9. REFERENCES

- [1] G. Chiarot and C. Silvestri, "Time series compression survey," *ACM Computing Surveys*, vol. 55, no. 10, 2023. [Online]. Available: <https://www.semanticscholar.org/reader/55f1751b726b35f935dfc68c0703d24b2b2c705e>
- [2] J. G. Matt, P. Huang, and B. Maag, "Lossless Compression of Time Series Data: A Comparative Study," *arXiv preprint*, arXiv:2510.07015v1, 2025. [Online]. Available: <https://arxiv.org/html/2510.07015v1>
- [3] S. Jamalidinan and K. Cheshmi, "Floating-Point Data Transformation for Lossless Compression," *arXiv preprint*, arXiv:2506.18062v1, 2025. [Online]. Available: <https://arxiv.org/html/2506.18062v1>
- [4] N. Azami, A. Fallin, and M. Burtscher, "Efficient Lossless Compression of Scientific Floating-Point Data on CPUs and GPUs," in *Proceedings of the 30th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*, 2025. [Online]. Available: <https://userweb.cs.txstate.edu/~burtscher/papers/asplos25.pdf>
- [5] W. Wang, W. Chen, Q. Lei, Z. Li, and H. Zhao, "ACTF: An efficient lossless compression algorithm for time series floating point data," *Journal of King Saud University – Computer and Information Sciences*, vol. 36, no. 102246, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157824003355>
- [6] P. Liakos, K. Papakonstantinou, and Y. Kotidis, "Chimp: Efficient Lossless Floating Point Compression for Time Series Databases," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 15, no. 11, pp. 3058–3070, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol15/p3058-liakos.pdf>

- [7] M. Burtscher and P. Ratanaworabhan, "FPC: A High-Speed Compressor for Double-Precision Floating-Point Data," *IEEE Transactions on Computers*, vol. 58, no. 1, pp. 18–31, 2009. [Online]. Available: <https://userweb.cs.txstate.edu/~burtscher/papers/tc09.pdf>
- [8] R. Li, Z. Li, Y. Wu, C. Chen, S. Guo, M. Zhang, and Y. Zheng, "Erasing-based lossless compression method for streaming floating-point time series," *arXiv preprint*, arXiv:2306.16053, 2023. [Online]. Available: <https://arxiv.org/pdf/2306.16053>
- [9] A. Afroozeh, L. Kuffó, and P. Boncz, "ALP: Adaptive Lossless Floating-Point Compression," *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD '24)*, 2024. [Online]. Available: <https://ir.cwi.nl/pub/33334/33334.pdf>
- [10] S. Chandak, K. Tatwawadi, C. Wen, L. Wang, J. Aparicio, and T. Weissman, "LFZip: Lossy compression of multivariate floating-point time series data via improved prediction," *CatalyzeX Research Repository*, 2019. [Online]. Available: <https://www.catalyzex.com/paper/lfzip-lossy-compression-of-multivariate>
- [11] C. E. Muñoz-Cuza, M. Boehm, and T. B. Pedersen, "CAMEO: Autocorrelation-Preserving Line Simplification for Lossy Time Series Compression," *arXiv preprint*, arXiv:2501.14432, 2025. [Online]. Available: <https://arxiv.org/pdf/2501.14432>
- [12] Z. Li, W. Wang, R. Li, C. Chen, and S. Guo, "Falcon: GPU-Based Floating-Point Adaptive Lossless Compression," *arXiv preprint*, arXiv:2511.04140, 2025. [Online]. Available: <https://arxiv.org/pdf/2511.04140>
- [13] S.-H. Hwang, K.-M. Kim, S. Kim, and J. W. Kwak, "Lossless Data Compression for Time-Series Sensor Data Based on Dynamic Bit Packing," *Sensors*, vol. 23, no. 20, p. 8575, 2023. [Online]. Available: <https://www.mdpi.com/1424-8220/23/20/8575>
- [14] A. Singh and R. Gupta, "Time Series Compression for IoT: A Systematic Literature Review," *Wiley Journal of Intelligent Systems*, 2023. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1155/2023/5025255>
- [15] A. Guerra, G. Cenedese, and S. Soatto, "Learned Compression of Nonlinear Time Series With Random Access," *ResearchGate Preprint*, 2024. [Online]. Available: https://www.researchgate.net/publication/387350813_Learned_Compression_of_Nonlinear_Time_Series_With_Random_Access